

passgen

Детерминированный генератор паролей

Полная документация проекта для защиты
Подробное описание всех функций программы

1. Идея проекта и решаемая проблема

В современном мире у каждого человека есть множество аккаунтов: электронная почта, социальные сети (ВКонтакте, Telegram), рабочие сервисы (GitHub), образовательные платформы, онлайн-банкинг, интернет-магазины. Для каждого требуется пароль. Использовать один и тот же пароль везде очень опасно: если злоумышленник узнает пароль от одного сервиса (например, из-за утечки базы данных), он получит доступ ко всем аккаунтам. Запоминать 20-30 разных сложных паролей человеческая память не позволяет.

Менеджеры паролей (LastPass, Bitwarden) хранят все пароли в одном зашифрованном месте, но это создает единую точку отказа. Если злоумышленник получит доступ к мастер-паролю от менеджера или если сам менеджер будет взломан, все пароли окажутся скомпрометированы.

passgen предлагает принципиально другой подход — детерминированную генерацию. Программа не хранит пароли, а каждый раз вычисляет их заново с помощью криптографического алгоритма HMAC-SHA256. Пользователю нужно запомнить всего одну фразу (сид). Пароль для любого сервиса восстанавливается по формуле: $\text{пароль} = \text{HMAC-SHA256}(\text{сид}, \text{сервис} + \text{длина} + \text{шаг})$.

Преимущества:

- Пароли не нужно хранить — они всегда восстанавливаются из сида и названия сервиса
- Нечего красть — на компьютере нет базы данных с паролями
- Пароль невозможно потерять — при знании сида он восстанавливается в любой момент
- Для каждого сервиса создается уникальный пароль
- Не требуется интернет
- Программа бесплатна и имеет открытый исходный код

2. Структура программы (index.html)

Вся программа находится в одном файле — index.html. Он состоит из трех частей: HTML (разметка страницы), CSS (стили и оформление) и JavaScript (логика работы). Никаких дополнительных файлов, библиотек или серверов не требуется. Файл открывается в любом современном браузере.

2.1. HTML — разметка страницы

Страница состоит из блоков (карточек), расположенных вертикально сверху вниз:

1. Верхняя панель с заголовком "passgen" и ссылками "Экспорт"/"Импорт".
2. Карточка "Сид" — поле для секретной фразы и место для сообщения об ошибке.
3. Карточка "Сервис" — поле ввода названия сервиса и кнопка "список".
4. Карточка "Параметры" — настройки генерации.
5. Кнопка "Создать".
6. Карточка "Логин и пароль" — результат генерации.
7. Кнопка "Копировать".
8. Модальное окно импорта (скрыто по умолчанию).

2.2. CSS — стили и оформление

Цвета задаются через CSS-переменные в блоке :root. Основные цвета: светло-фиолетовый фон страницы, белые карточки, темный текст, фиолетовый акцентный цвет. Кнопки имеют градиентную заливку, карточки — скругленные углы и тень. Реализованы анимации: плавное появление элементов (fadeUp), пульсация границы при ошибке (glow), раскрытие панелей (slideDown/slideUp), переключение кружочков (checkPop).

2.3. JavaScript — логика программы

На JavaScript написана вся логика программы. Используется Web Crypto API — встроенная в браузер криптографическая библиотека, доступная в Chrome, Firefox, Edge, Safari. Она позволяет вычислять HMAC-SHA256 и SHA-256 прямо на устройстве пользователя без отправки данных в интернет. В коде реализованы: генерация паролей, сохранение и восстановление настроек для каждого сервиса, экспорт и импорт настроек в JSON, обработка всех событий (клики, ввод текста, клавиши).

3. Криптографический алгоритм

3.1. Что такое HMAC-SHA256

HMAC-SHA256 — это криптографический алгоритм, который combines в себе две вещи: хеш-функцию SHA-256 и секретный ключ. HMAC расшифровывается как Hash-based Message Authentication Code — код аутентификации сообщения на основе хеша. SHA-256 — это стандартная криптографическая хеш-функция, которая преобразует любые входные данные в 256-битное (32-байтовое) значение фиксированной длины.

Чем HMAC отличается от обычного SHA-256? Обычный SHA-256 просто берет входные данные и вычисляет их хеш. Это работает как отпечаток пальца: по хешу нельзя восстановить исходные данные, но одинаковые данные дают одинаковый хеш. Проблема в том, что любой человек может вычислить SHA-256 от любых данных — для этого не нужен секрет.

HMAC решает эту проблему, добавляя секретный ключ в процесс хеширования. Алгоритм смешивает ключ с данными по специальной схеме (два прохода хеш-функции с разными вариантами ключа). Результат HMAC невозможно вычислить без знания ключа. Даже если злоумышленник узнает результат HMAC, он не сможет восстановить ключ или подделать результат для других данных. В программе passgen сид пользователя используется именно как такой секретный ключ.

Простыми словами: если SHA-256 — это "черный ящик", который превращает любую строку в набор символов, то HMAC-SHA256 — это "сейф", который открывается только специальным ключом (сидом). Без ключа вычислить пароль невозможно. С ключом — легко и всегда одинаково.

3.2. Пошаговая схема генерации пароля

Шаг 1. Пользователь вводит сид (секретную фразу). Программа преобразует эту фразу в набор чисел (байтов) с помощью функции `TextEncoder`. Из этих байтов создается криптографический ключ через функцию `crypto.subtle.importKey`.

Шаг 2. Формируется сообщение для алгоритма. Программа берет название сервиса (например, "google"), добавляет к нему желаемую длину пароля (например, 16) и номер шага. Номер шага начинается с 0 и увеличивается каждый раз, когда программе нужно больше случайных данных. Все это объединяется в одну строку и превращается в байты.

Шаг 3. Вычисляется HMAC-SHA256. Программа вызывает `crypto.subtle.sign("HMAC", key, message)`. Результат — 32 байта криптостойких случайных данных. Эти данные невозможно предсказать без знания ключа (сида).

Шаг 4. Превращение байтов в символы пароля. Из 32 полученных байтов каждый проверяется: если его значение меньше максимального порога (для равномерного распределения), он превращается в символ из выбранного пользователем алфавита. Если байт не подходит — он пропускается. Если в одном блоке не хватило подходящих байтов, номер шага увеличивается, и вычисляется следующий блок. Процесс повторяется, пока не наберется нужное количество символов.

Шаг 5. Применяются опции инверсии. Если пользователь включил "Первую заглавную" — первый символ пароля делается заглавным. Если включил "Последнюю цифру" — последний символ заменяется на цифру, которая вычисляется отдельным HMAC, чтобы не нарушить детерминированность основного пароля.

3.3. Rejection Sampling — обеспечение равномерного распределения

При превращении байтов HMAC в символы пароля возникает математическая проблема. Каждый байт может принимать значение от 0 до 255 (всего 256 значений). Длина алфавита может быть, например, 50 или 62 символа. Число 256 не делится нацело ни на 50, ни на 62.

Если бы программа просто брала остаток от деления байта на длину алфавита (байт % длина), то некоторые символы встречались бы чаще других. Например, для алфавита из 50 символов: байты 0-49, 50-99, 100-149, 150-199, 200-249 дают равномерное распределение (каждый символ по 5 раз). Но байты 250-255 (всего 6 штук) дают символы 0-5 дополнительно. В итоге символы 0-5 встречаются 6 раз, а символы 6-49 — только 5. Это смещение (bias) недопустимо в криптографии.

Решение: программа вычисляет максимальное допустимое значение — наибольшее число, кратное длине алфавита, но не превышающее 256. Формула: $\text{max} = 256 - (256 \% \text{длина_алфавита})$. Байты, значение которых больше или равно max, просто отбрасываются (reject). Используются только байты меньше max. Поскольку max кратен длине алфавита, распределение символов получается строго равномерным.

Пример: алфавит из 50 символов. $\text{max} = 256 - (256 \% 50) = 256 - 6 = 250$. Байты 250-255 отбрасываются. Байты 0-249 дают ровно $50 * 5 = 250$ значений. Каждый символ алфавита встречается ровно 5 раз. Идеальное равномерное распределение.

Этот метод называется Rejection Sampling (отбраковочная выборка). Он является стандартной техникой в криптографии и используется во многих библиотеках генерации паролей, включая известные менеджеры паролей.

3.4. Алфавиты наборов символов

Каждый набор символов содержит строго определенный список. Из некоторых наборов исключены потенциально путающиеся символы.

Заглавные: ABCDEFGHJKLMNPQRSTUVWXYZ (исключены I и O — путаются с 1 и 0).

Строчные: abcdefghikmnpqrstuvwxy (исключены j, l, o — путаются с цифрами).

Цифры: 0123456789

Спецсимволы: !@#\$%&*+=-.

По умолчанию включены: заглавные, строчные, спецсимволы. Длина 10 символов.

3.5. Генерация логина

Логин генерируется отдельно от пароля, через алгоритм SHA-256 (без ключа, просто хеш). Программа берет сид, добавляет к нему слово "login", название сервиса и значение счетчика. Счетчик увеличивается при каждом вызове функции, поэтому каждый клик по кнопке "Создать логин" дает новый логин.

Из 32 байтов хеша берутся первые 6 и превращаются в символы из специального алфавита: "abcdefghijklmnopqrstuvwxyz23456789" (32 символа). Из алфавита исключены буквы i, l, o (их можно перепутать с цифрами 1 и 0), а также цифры 0 и 1 (их можно перепутать с буквами).

4. Интерфейс программы

Подробное описание каждого элемента пользовательского интерфейса.

4.1. Поле "Сид"

Это самое важное поле во всей программе. Сюда пользователь вводит свою секретную фразу — единственное, что нужно запомнить. Без сида невозможно восстановить пароль. Если сид не введен, при попытке генерации поле подсвечивается розовой пульсирующей границей (анимация glow) и появляется сообщение "Введите сид". При начале ввода ошибка исчезает. Нажатие клавиши Enter в этом поле запускает генерацию пароля.

4.2. Поле "Сервис" и список сервисов

В это поле вводится название сайта или сервиса: google, github, vk, yandex и так далее. Справа от поля находится кнопка "список" со стрелкой, которая открывает панель со списком сервисов. В панели отображаются популярные сервисы с цветными кружками-логотипами, а также сервисы, которые пользователь уже использовал ранее.

Популярные сервисы: Google (синий G), GitHub (черный GH), VK (синий V), Яндекс (красный Ya), Mail (синий @), iCloud (темно-синий i), Onliner (оранжевый O), Kufar (зеленый K).

При клике на сервис из списка его название подставляется в поле, а сохраненные настройки (длина, символы, инверсия) автоматически восстанавливаются. Если нужного сервиса нет в списке, можно ввести его название вручную — после генерации пароля он автоматически добавится в список. Правый клик на сохраненном сервисе удаляет его из списка.

4.3. Параметры генерации

Выпадающий список пресетов: "Минимальный (8, буквы+цифры)", "Стандартный (10, все кроме спецсимволов)", "Максимальный (16, все)". Пункт "Свой" означает ручную настройку. Длина пароля от 1 до 99. Четыре кружочка — наборы символов. Фиолетовый = включено. Расширенные настройки: "Первая заглавная" и "Последняя цифра" (скрыты по умолчанию).

4.4. Кнопка "Создать"

Главная кнопка. Запускает полный цикл: проверка сида и чекбоксов, HMAC-SHA256, инверсия, отображение, копирование, добавление в список, сохранение. Enter в полях сида/сервиса равносителен нажатию. Фиолетовый градиент с тенью.

4.5. Результат

Логин (только чтение) + "Создать логин". Пароль (только чтение) — моноширинный, жирный, по центру. Индикатор: 3 полосы, цвет от розового до сиреневого, рекомендация.

4.6. Кнопка "Копировать"

Копирует пароль в буфер. Clipboard API, при ошибке exesCommand. "Скопировано!" на 1.5с.

4.7. Экспорт и импорт

Экспорт: JSON со всеми сервисами и настройками.

Импорт: загружает JSON, окно выбора с чекбоксами.

5. Все функции JavaScript (подробное описание)

В этом разделе подробно описывается каждая функция программы. Описания написаны человеческим языком, без использования программного кода, чтобы их можно было устно пересказать на защите проекта.

5.0. Схема вызова функций при генерации пароля

Ниже показана последовательность вызова функций в процессе генерации пароля. Стрелка (->) означает "вызывает".

```

Пользователь нажимает "Создать" или Enter
|
v
gen()
|-- проверяет сид (если пусто -> ошибка, выход)
|-- проверяет чекбоксы (если пусто -> ошибка, выход)
|-- составляет алфавит из выбранных наборов
|-- вызывает hmacGen(seed, svc, len, alpha)
|   |-- importKey (сид -> ключ HMAC)
|   |-- цикл: sign (HMAC) -> отбор байтов -> сбор символов
|   |-- возвращает пароль
|
|-- применяет инверсию (первая заглавная, последняя цифра)
|-- записывает пароль в поле pwd
|-- вызывает updateStrength(len, sets)
|   |-- сравнивает с порогами (14/4, 10/3)
|   |-- рисует полосы и текст
|
|-- вызывает сру() [копирование в буфер]
|-- добавляет сервис в svcs (если новый)
|-- вызывает renderSvcChips() [перерисовка списка]
|-- вызывает saveCfg() [сохранение настроек]
    |-- perSvc[сервис] = {state, inv, len}

```

Схема для генерации логина (кнопка "Создать логин"):

```

genLogin()
|-- проверяет сид (если пусто -> ошибка, выход)
|-- проверяет сервис (если пусто -> выход)
|-- loginCounter++
|-- digest(SHA-256, seed+"login"+svc+counter)
|-- 6 байтов -> символы из алфавита логина
|-- записывает логин в поле login

```

5.1. Функция `init()` — подготовка программы к работе

Функция запускается автоматически, когда пользователь открывает страницу программы. Она подготавливает все элементы интерфейса к работе — устанавливает начальные значения, привязывает обработчики событий к полям и кнопкам.

Сначала `init()` вызывает `loadState()` для очистки всех данных, которые могли остаться от предыдущего использования. Затем убирает ошибки с поля сида — снимает розовую подсветку и очищает текст сообщения об ошибке.

Поле сида получает обработчик `input`: при каждом вводе текста ошибка снимается. Поля сида и сервиса получают обработчик `keydown`: при нажатии `Enter` запускается генерация пароля. Поле сервиса получает обработчик `blur`: при потере фокуса программа запоминает, какой сервис выбран, и если для него есть сохраненные настройки — применяет их.

Затем `init()` проверяет, есть ли сохраненные настройки для текущего сервиса. Если есть — восстанавливает их. Если нет — устанавливает стандартные: заглавные+строчные+специмволы, длина 10, инверсия выключена. В конце отрисовывает список сервисов с логотипами.

5.2. Функция loadState() — очистка всех данных

Полностью очищает все данные программы, чтобы каждый запуск начинался с чистого листа. Это ключевая функция для обеспечения безопасности.

Удаляет данные из localStorage. Обнуляет объект настроек perSvc. Очищает массив сервисов svcs. Сбрасывает текущий сервис на "(empty)". Устанавливает стандартные настройки, длину 10. Очищает поле ввода сервиса.

Благодаря этой функции, если закрыть программу и открыть снова, никаких следов предыдущего использования не останется.

5.3. Функция `saveCfg()` — сохранение настроек для сервиса

Запоминает текущие настройки (чекбоксы, длину, инверсию) и сохраняет их для текущего сервиса. При повторном выборе того же сервиса настройки восстанавливаются автоматически.

Берет состояние чекбоксов из `window.state`, опции инверсии из `window.inv`, длину из поля. Упаковывает в объект и сохраняет в `perSvc[сервис]`. Настройки только в оперативной памяти — при закрытии страницы исчезают.

5.4. Функция `applyCfg()` — восстановление настроек сервиса

Применяет сохраненные настройки для выбранного сервиса к интерфейсу. Принимает объект `cfg`, устанавливает длину, восстанавливает чекбоксы и инверсию, перерисовывает интерфейс.

5.5. Функция `toggleInv()` — переключение расширенных опций

Переключает "Первую заглавную" или "Последнюю цифру" при клике. Меняет 0 на 1 и наоборот в `window.inv`. Обновляет кружок. Не сохраняет настройки — сохранение только при "Создать".

5.6. Функция `applyPreset()` — применение готового пресета

Устанавливает готовую комбинацию настроек. Минимальный: длина 8, caps+lower+digits. Стандартный: длина 10, caps+lower+digits. Максимальный: длина 16, все 4 набора. "Свой" — без действий. После применения перерисовывает чекбоксы и сохраняет настройки.

5.7. Функция renderChk() — отрисовка чекбоксов символов

Создает четыре строки с кружочками и названиями наборов. Активный набор — фиолетовый кружок (класс "on"). Клик переключает и перерисовывает.

5.8. Функция `renderInv()` — отрисовка опций инверсии

Обновляет кружочки для `cap` и `dig`: добавляет или убирает класс `"on"`.

5.9. Функция `adj()` — изменение длины пароля

+1 или -1 по стрелкам. Проверяет границы 1-99.

5.10. Функция toggleSvcPanel() — открытие и закрытие списка сервисов

Переключает класс "open" на панели. max-height 0 <-> 300px. Стрелка поворачивается.

5.11. Функция `renderSvcChips()` — сборка списка сервисов

Сначала популярные (которых нет в `svcs`), потом все сохраненные сервисы из `svcs`. Для каждого вызывает `chip()`.

5.12. Функция `chip()` — создание кнопки сервиса с логотипом

`item` может быть объектом (популярный) или строкой (пользовательский). Цветной кружок с буквой или серый с "?". Клик: заполняет поле, применяет настройки. ПКМ: удаляет.

5.13. Функция `gen()` — генерация пароля (главная функция)

Запускается при нажатии "Создать" или Enter. Выполняет 10 шагов:

1. Проверка сида — если пусто, розовая подсветка, "Введите сид", останов.
2. Проверка чекбоксов — если ни один не включен, ошибка, останов.
3. Составление алфавита — из выбранных наборов.
4. `hmacGen()` — асинхронное вычисление HMAC-SHA256.
5. Инверсия: первая заглавная (`toUpperCase`), последняя цифра (отдельный HMAC).
6. Отображение пароля в поле `pwd`.
7. `updateStrength(len, sets)` — индикатор надежности.
8. `cpy()` — копирование в буфер обмена.
9. Добавление сервиса в `svcs`, сортировка, `renderSvcChips()`.
10. `saveCfg()` — сохранение настроек.

5.14. Функция hmacGen() — криптографическое ядро программы

Выполняет генерацию пароля через HMAC-SHA256. Асинхронная (не блокирует браузер).

seed -> байты -> ключ через importKey. Сообщение: svc+len в байтах. max = 256 - (256 % alpha.length) для rejection sampling.

Цикл: пока pwd.length < len:

- добавить счетчик ct к сообщению
- crypto.subtle.sign("HMAC", key, message) -> 32 байта
- для байт < max: символ = alpha[байт % alpha.length]
- ct++

Возвращает пароль.

5.15. Функция genLogin() — генерация логина

6-символьный логин. Проверяет сид и сервис. loginCounter++. SHA-256 от seed+"login"+svc+счетчик. Первые 6 байтов -> символы из "abcdefghijklmnopqrstuvwxyz23456789" (без i,l,o,0,1). Каждый клик — новый.

5.16. Функция `updateStrength()` — индикатор надежности пароля

`len` $\geq$ 14 и `sets` $\geq$ 4: отличный (сиреневый, 3 полосы, "Отличный пароль"). `len` $\geq$ 10 и `sets` $\geq$ 3: средний (малиновый, 2, "Хороший, можно длиннее"). Иначе: слабый (розовый, 1, "Добавьте символов или увеличьте длину").

5.17. Функция `сру()` — копирование пароля в буфер обмена

`navigator.clipboard.writeText()`, при ошибке `execCommand("copy")`. "Скопировано!" на 1.5 секунды.

5.18. Функция `exportData()` — экспорт настроек в JSON

Собирает `perSvc`, формирует `data = {svcs, perSvc}`, `JSON.stringify` с отступами, `Blob`, ссылка, `download = "passgen_export.json"`.

5.19. Функция `importData()` — импорт настроек из JSON

FileReader, парсинг JSON, проверка полей `perSvc/svcs`, сохранение в `importDataCache`, `showImportModal()`.

5.20. Функция `showImportModal()` — окно выбора сервисов

Список сервисов с чекбоксами (`checked=true`). Если пусто — "Нет сервисов для импорта".

5.21. Функция `confirmImport()` — подтверждение импорта

Отмеченные чекбоксы -> `perSvc[svc]` = данные, добавление в `svcs`, сортировка, перерисовка. "Импорт завершен!" на 3 секунды.

5.22. Функция `closeImportModal()` — закрытие окна импорта

`remove class "open", importDataCache = null.`

5.23. Функция `toggleAdvanced()` — открытие расширенных настроек

`max-height 0 <-> 120px`, `opacity 0 <-> 1`. Стрелка поворачивается.

5.24. Функция `getCfg()` — получение настроек сервиса

Если `perSvc[svc]` нет — создает стандартные. Возвращает настройки.

5.25. Функция onSvcChange() — обработчик смены сервиса

curSvc = значение поля, если есть настройки — applyCfg().

5.26. Функция saveState() — заглушка для безопасности

Пустая. Ранее сохраняла в localStorage. Оставлена для совместимости.

6. Обработка ошибок

1. Пустой сид — розовая подсветка, "Введите сид". При вводе исчезает.
2. Нет набора символов — "Выберите хотя бы один тип символов".
3. Длина вне 1-99 — принудительная граница.
4. Неверный файл импорта — сообщение об ошибке.
5. В файле нет сервисов — "Нет сервисов для импорта".
6. Clipboard недоступен — exesCommand как запасной.

7. Экспорт и импорт (формат JSON)

Пример структуры экспортируемого/импортируемого файла:

```
{
  "svcs": ["google", "github", "vk"],
  "perSvc": {
    "google": {
      "state": {"capitals":1,"lowercase":1,"digits":0,"symbols":1},
      "inv": {"cap":0,"dig":1},
      "len": 16
    }
  }
}
```

svcs — массив сервисов. perSvc — настройки для каждого: state (флаги capitals, lowercase, digits, symbols), inv (cap — первая заглавная, dig — последняя цифра), len — длина пароля.

8. Безопасность

1. Локальность. Все вычисления происходят на локальном компьютере в браузере. Данные не отправляются в интернет и не могут быть перехвачены.
2. Сессионный режим. Настройки только в RAM, при перезагрузке все удаляется.
3. Стандартная криптография. HMAC-SHA256 (RFC 2104). Используется в банках, TLS, blockchain.
4. Rejection Sampling. Равномерное распределение символов, без modulo bias.
5. Необратимость. Зная пароль, нельзя восстановить сид.
6. Разные пароли — название сервиса входит в HMAC.

Ограничения

Не защищает от кейлоггеров. Не требует мастер-пароля (сид = мастер-пароль). Сид нужно хранить отдельно.

9. Запуск программы

Программа представляет собой один файл index.html. Способ запуска:

Открыть файл index.html в любом современном браузере (Chrome, Firefox, Edge, Safari, Opera) двойным кликом или через меню браузера (Ctrl+O -> выбрать файл).

Системные требования:

- Любой современный браузер
- Web Crypto API (встроен во все современные браузеры)
- Интернет не требуется